



## Trial final exam with solutions

Introduction to python (Universiteit van Amsterdam)



messages.pdf\_cover\_qr\_code\_label

## Question 1 – 6–OOP–1 – 307795.1.0

You'd like to define a class called **Vehicle**. It should have two attributes: "name" and "mode", which you want to be initialized at the time of object construction.

The "name" attribute can be any string, and it must be passed to the object constructor. The "mode" attribute is also a string, but it doesn't necessarily need to be passed to the object constructor. **If it isn't specified at object construction**, then the "mode" attribute should be equal to "land".

For example, if we create a **Vehicle** object as:

```
my_car = Vehicle("Mazda")
```

then the following two conditions should both be true:

```
my_car.name == "Mazda"
```

```
my_car.mode == "land"
```

Which of the following code segments achieves what you want?

- A**
- ```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode="land"):
        self.name = vehicle_name
        self.mode = vehicle_mode
```
- B**
- ```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode):
        self.name = vehicle_name
        self.mode = vehicle_mode
```
- C**
- ```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode="land"):
        return name, mode
```
- D**
- ```
class Vehicle:
    def __init__(vehicle_name, vehicle_mode):
        name = vehicle_name
        mode = vehicle_mode
```

## Question 2 – 6–OOP–2 – 307818.1.1

Take a look at the following class definition.

```
class Book:
    def __init__(self, title, author, review_scores=None):
        self.title = title
        self.author = author
        self.review_scores = review_scores if review_scores else []

    def add_review(self, score):
        self.review_scores.append(score)

    def show_rating(self):
        if self.review_scores:
            return round(sum(self.review_scores) / len(self.review_scores), 1)
```

Suppose that we create a **Book** object and call its **add\_review** method three times, like this:

```
book_1 = Book("The Lightning Thief", "Rick Riordan")
book_1.add_review(5)
book_1.add_review(3)
book_1.add_review(3)
```

What will then the following command print to the screen?

```
print(book_1.show_rating())
```

- A** 3.7
- B** 4
- C** None
- D** 3.67

## Question 3 – 6–strings–1 – 306997.1.1

Imagine that you have a list called **students**, which is populated by a number of dictionaries.

Each dictionary in the **students** list has two keys: "Name" and "Grade". The corresponding values are student names (as strings) and course grades (as floats).

For each student, you'd like to print the following text to the screen:

```
[NAME] has received a grade of [GRADE].
```

where [NAME] is substituted by the name of the student and [GRADE] is substituted by the actual course grade. The course grade should be displayed to **one decimal** place after the decimal point.

For example, if

```
students = [{"Name": "Adam", "Grade": 7.5}, {"Name": "Bernard", "Grade": 8.0}]
```

then you'd like to see the following two lines printed to the screen:

```
Adam has received a grade of 7.5.
```

```
Bernard has received a grade of 8.0.
```

Which of the following code segments will achieve what you want?

- A**

```
for student in students:
    print(f'{student['Name']} has received a grade of {student['Grade']:.1f}.')
```
- B**

```
for student, grade in students.items():
    print(f'{student} has received a grade of {grade:.1f}.')
```
- C**

```
for student in students:
    print(f'{student["Name"]} has received a grade of {student["Grade"]:.1f}.')
```
- D**

```
for student, grade in students:
    print(f'{student} has received a grade of {grade:.1f}.')
```

#### Question 4 – 6-strings-2 – 307020.1.0

You need to write a function called **format\_phone\_number** that formats phone numbers. The output of the function should be a string that represents a 10-digit number. The input argument is also a string with 10 digits, but the digits may be separated by dashes (the "-" character) at various places.

For example, the following function calls:

```
format_phone_number("020-525-1400")  
format_phone_number("020-5251400")  
format_phone_number("020-525-14-00")
```

should all return the string:

```
"0205251400"
```

Which of the following code segments does **not** achieve what you want?

- A**

```
def format_phone_number(number):  
    number.replace("-", "")  
    return number
```
- B**

```
def format_phone_number(number):  
    return number.replace("-", "")
```
- C**

```
def format_phone_number(number):  
    digits = []  
    for char in number:  
        if char in "0123456789":  
            digits.append(char)  
    return "".join(digits)
```
- D**

```
def format_phone_number(number):  
    return "".join(number.split("-"))
```

#### Question 5 – 7-pandas-new\_column-1 – 306976.1.0

You have a pandas DataFrame called **df**. It has a column called "A" and a column called "B". Both contain numbers.

You'd like to create a column called "C", which contains the sum of the numbers in columns "A" and "B". Which of the following code lines will achieve what you want?

- A**

```
df["C"] = df["A"] + df["B"]
```
- B**

```
df["C"] = df["A"].map(lambda x: x + df["B"])
```
- C**

```
df["C"] = df.columns("A" + "B")
```
- D**

```
df["C"] = df["A" + "B"]
```

### Question 6 – 7-pandas-selection-1 – 306978.1.0

You have a pandas DataFrame called **df** that looks like this, when printed:

|   | A   | B   | C   | D   |
|---|-----|-----|-----|-----|
| 1 | 9.1 | 9.5 | 9.0 | 6.0 |
| 2 | 6.4 | 8.2 | 7.5 | 1.7 |
| 3 | 2.5 | 5.3 | 8.8 | 3.0 |
| 4 | 6.7 | 3.6 | 7.2 | 6.9 |
| 5 | 8.4 | 1.9 | 8.2 | 3.3 |
| 6 | 8.1 | 8.4 | 9.9 | 2.2 |

Suppose that you'd like to create a new DataFrame that only contains those elements of column "B" in **df** that have an even-numbered index. Which of the following code lines will achieve what you want?

- A** `df.loc[df.index % 2 == 0, ["B"]]`
- B** `df[2, 4, 6, "B"]`
- C** `df.iloc[[2, 4, 6], 2]`
- D** `df.loc[2::2, "B"]`

### Question 7 – 7-pandas-sorting-1 – 306979.1.0

There are two pandas DataFrames: **df1** and **df2**. They look like this, when printed:

```
print(df1)
```

|   | A   | B   | C   | D   |
|---|-----|-----|-----|-----|
| 1 | 5.0 | 8.2 | 9.5 | 4.4 |
| 2 | 6.0 | 2.4 | 9.1 | 3.3 |
| 3 | 3.2 | 5.2 | 8.9 | 3.6 |
| 4 | 7.8 | 8.7 | 7.9 | 8.0 |
| 5 | 1.2 | 9.7 | 2.9 | 3.0 |

```
print(df2)
```

|   | A   | C   | D   |
|---|-----|-----|-----|
| 4 | 7.8 | 7.9 | 8.0 |
| 3 | 3.2 | 8.9 | 3.6 |
| 2 | 6.0 | 9.1 | 3.3 |

Which of the following code lines could **not** have created **df2** from **df1**?

- A** `df2 = df1.sort_index(ascending=False)[["A", "C", "D"]]`
- B** `df2 = df1.iloc[[3, 2, 1], [0, 2, 3]]`
- C** `df2 = df1.loc[2:4, ["A", "C", "D"]].sort_values("D", ascending=False)`
- D** `df2 = df1.loc[2:4, ["A", "C", "D"]].sort_values("C")`

### Question 8 – 7-pandas-string-1 – 306977.1.0

There is a pandas Series called **s**, which contains Dutch municipality and corresponding province names, separated by a semicolon and a whitespace. The following, for example, is an element of this Series: "Amsterdam; Noord-Holland".

You'd like to create a new Series that only contains the municipality names from **s** in the same order. Which of the following code lines will achieve what you want?

- A** `s.map(lambda x: x.split("; ")[0])`
- B** `s.split("; ")[0]`
- C** `s.lambda(x.split("; "))`
- D** `s.str[:9]`

### Question 9 – 8-datetime-1 – 306982.1.1

Take a look at the following code segment.

```
from datetime import datetime

def get_months(date_strings):
    dates = []
    for date_str in date_strings:
        if "-" in date_str:
            dates.append(datetime.strptime(date_str, "%d-%m-%Y"))
        elif "/" in date_str:
            dates.append(datetime.strptime(date_str, "%m/%d/%Y"))
        else:
            dates.append(None)
    return [date.month if date is not None else None for date in dates]
```

Suppose that you call the function like this:

```
get_months(["03/02/2013", "03.02.2013", "03-02-2013"])
```

What does the function call return?

- A** `[3, None, 2]`
- B** `[2, None, 3]`
- C** `[3, 2]`
- D** `[2, 3]`

### Question 10 – 8-datetime-2 – 306983.1.0

Suppose that you want to write a function called **day\_of\_year** that takes a datetime value and returns the number of the day that the datetime value represents within the year. The return value is therefore always an integer between 1 and 366.

For example, the function call:

```
day_of_year(datetime(2022, 2, 1))
```

should return the integer:

32

Which of the following code segments will achieve what you want? (You can assume that the datetime and timedelta classes are already imported from the datetime module.)

- A**

```
def day_of_year(dt):  
    return (dt - datetime(dt.year, 1, 1)).days + 1
```
- B**

```
def day_of_year(dt):  
    return timedelta(dt - datetime(dt.year, 1, 1)) + 1
```
- C**

```
def day_of_year(dt):  
    return dt - datetime(dt.year, 1, 1) + 1
```
- D**

```
def day_of_year(dt):  
    return (dt - datetime(dt.year, 1, 0)).days
```

### Question 11 – 8-dict\_comprehension-1 – 306981.1.1

You have a list of strings called **list\_1**, in which every element is unique.

Which of the following code lines will create a dictionary, in which the keys are the elements of **list\_1** and the values are the number of characters in the corresponding key?

- A**

```
{item: len(item) for item in list_1}
```
- B**

```
[len(item) for item in list_1]
```
- C**

```
{item for item in list_1 if len(item) > 0}
```
- D**

```
{len(item): item for item in list_1}
```

### Question 12 – 8-list\_comprehension-1 – 306980.1.0

Suppose that you have a list of letters called **list\_1**. Some letters are upper case, others are in lower case.

You'd like to create another list that contains the same letters in the same order, but you want to turn every upper case letter into its lower case version, and every lower case letter into its upper case version.

Which of the following code lines will achieve what you want?

- A**

```
[letter.upper() if letter.islower() else letter.lower() for letter in list_1]
```
- B**

```
[letter.upper() for letter in list_1 if letter.islower() else letter.lower()]
```
- C**

```
[letter.lower() for letter in list_1 if letter.isupper()] + [letter.upper() for letter in list_1 if letter.islower()]
```
- D**

```
[letter.capitalize() for letter in list_1 if not letter.iscapitalized()]
```

### Question 13 – Basic logic – 306991.2.0

Suppose you have three variables **x**, **y**, **z**, as follows:

```
x = '3'  
y = 3  
z = 3.0
```

What are the outputs of the following lines of code?

```
print(x == y)  
print(y == z)  
print(x == z)  
print(x == y == z)
```

**A**    False  
      True  
      False  
      False

**B**    True  
      True  
      True  
      True

**C**    False  
      False  
      False  
      True

**D**    True  
      False  
      False  
      False

### Question 14 – Basic loop – 306994.1.5

You need to write a simple loop that iterates over the integers from 1 to 10, and in each iteration, prints the sum of the current and the previous number. (In the first iteration, just take the previous number to be 0.)

For example:

```
1 + 0 = 1  
2 + 1 = 3  
3 + 2 = 5  
...  
10 + 9 = 19
```

Choose the program that will print the correct output (i.e.: only the sums).

**A**    previous = 0  
      for i in range(1, 11):  
          print(i + previous)  
          previous = i

**B**    i = 0  
      while i <= 10:  
          print(i + (i-1))  
          i += 1

**C**    Both of the above

**D**    Neither of the above



### Question 15 – Basic scope – 306996.1.3

Executing the following program will produce an error. Why?

```
def cube(num):  
    result = num ** 3  
    return f'The cube of {num} is: {result}.'  
print(result)
```

- ☒ **A** Result is a local variable and cannot be referenced outside of the function.
- ☐ **B** \*\* is not a valid Python operator.
- ☐ **C** The syntax of the f-string is incorrect.
- ☐ **D** You can only use either `print` or `return`, but not both.

### Question 16 – List lambda – 302507.3.3

You already have a list named **list\_1** with 2 lambda functions as follows:

```
list_1 = [lambda a, b: a + b, lambda a, b: a * b]
```

What would be printed by the following code snippet:

```
print(list_1[0](1, 2) ** list_1[1](1, 2))
```

- ☐ **A** 6
- ☐ **B** 7
- ☐ **C** 8
- ☒ **D** 9

### Question 17 – List logic – 306789.2.2

You have the following function called **main**:

```
def main(x):  
    y = []  
    for i in x:  
        if len(i) == 0:  
            y.append(True)  
        else:  
            y.append(False)  
    return y[0] == y[1] == y[2]
```

What are the outputs of the following lines:

```
main([[], {}, {}])  
main([[1], [2, 2], [3, 3, 3]])  
main([[1], [2], [3]])
```

- A** True  
True  
True
- B** True  
False  
True
- C** True  
False  
False
- D** False  
False  
True

### Question 18 – List loop – 306785.1.5

How do you break a list into a list of lists, in which a sub-list consists of 3 values each?

For example:

If you have the original list **x** as:

```
x = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
```

then how do you create:

```
y = [['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i']]
```

- A** `y = [x[i:i + 3] for i in range(0, 7, 3)]`
- B** `y = [x[i[0]::3] for i in enumerate(x)]`
- C** Both of the above work
- D** Neither of the above works

### Question 19 – trial-dictionary-1 – 307013.4.2

You have to write a function called **main** that accepts a dictionary as an input argument. Both the keys and the values of the input are integers.

Return another dictionary that has the same keys as the input argument. For each output key, the corresponding value should be calculated as the sum of those values from the input dictionary, whose keys are smaller than, or equal to, the output key in question.

For example:

```
main({5: 1, 2: 5, 4: 2})
```

should return a dictionary that is equal to:

```
{5: 8, 2: 5, 4: 7}
```

Which of the following programs works as intended?

- A**

```
def main(d1):  
    running_total = 0  
    d2 = {}  
    for key in sorted(d1.keys()):  
        running_total += d1[key]  
        d2[key] = running_total  
    return d2
```
- B**

```
def main(d):  
    result = {}  
    for output_key in d.keys():  
        result[output_key] = sum([v for k, v in d.items() if k <= output_key])  
    return result
```
- C** Both of the above work as intended
- D** None of the above work as intended

### Question 20 – trial-dictionary-2 – 307620.2.3

What is the result of the following program?

```
l1 = [1, 2, 3, 4]  
l2 = [2, 3, 4, 5]  
  
d1 = {}  
for key, value in zip(l1, l2):  
    d1[key] = value  
  
d2 = {}  
for key, value in enumerate(l2, 1):  
    d2[key] = value  
  
print(d1 == d2)
```

- A** True
- B** False
- C** An error message.
- D** None of the above

### Question 21 – trial-dictionary-3 – 307635.3.3

```
d1 = {1: 10, 2: 20, 3: 30, 4: 40}
```

The following codes all print the same value, except one. Which one?

- A**

```
result = 0
for key, value in d1:
    result += value
print(result)
```
- B**

```
result = 0
for key in d1.keys():
    result += d1[key]
print(result)
```
- C**

```
result = 0
for value in d1.values():
    result += value
print(result)
```
- D**

```
result = sum(d1.values())
print(result)
```

### Question 22 – trial-functions-1 – 301763.2.5

You have the following string

```
x = 'Amsterdam'
```

Which script, using the `count` string method, would print the following value:

1

- A**

```
print(count(x, 'a'))
```
- B**

```
print(x.count('a'))
```
- C** Both scripts would deliver that result
- D** Neither script would deliver that result

### Question 23 – trial-functions-2 – 307618.3.1

Take a look at the following function definition.

```
def calculation(func, *args):  
    result = 0  
    for el in args:  
        result += func(el)  
    return result
```

Which of the following function calls would return the number 30 as a result?

- ☐ A calculation(lambda x: x + 5, 1, 2, 3, 4)
- ☐ B calculation(lambda x: x\*\*2, 1, 2, 3, 4)
- ☒ C Both function calls would lead to that result
- ☐ D Neither function call would lead to that result

### Question 24 – trial-functions-3 – 301817.2.3

If you know the radius  $r$  of a circle and you want to calculate its area, you need the value of  $\pi$  from the math package and apply the formula:  $\pi r^2$

Suppose that the variable  $r$  already contains the radius of the circle. Which script does **not** print the correct answer for the area?

- ☐ A 

```
import math  
print(math.pi * r ** 2)
```
- ☐ B 

```
from math import pi  
print(pi * r ** 2)
```
- ☒ C 

```
import math as constants  
print(math.pi * r ** 2)
```
- ☐ D 

```
from math import pi as constant  
print(constant * r ** 2)
```